Evaluation of LLMs for Mathematical Formalization in Lean

Drew Bladek, Escher Crawford, Ariel Fu, Kaira Nair, Bohao Chen, Tyson Klingner*
Department of Mathematics, University of Washington (= Mentor)*

UNIVERSITY of WASHINGTON

MATH AI LAB
ai.math.uw.edu

Background

Proving mathematical theorems often requires a complex chain of arguments, logical deductions, and involves tedious computations. Hence, evaluating a Large Language Model's (LLMs) ability to produce mathematical proofs provides an important benchmark to measure a given model's reasoning ability. To do this, we can task LLMs to produce a *formal language* proof in a language such as Lean 4.

With an abundance of readily available LLMs it is natural to ask which LLM is the best at formal proof generation. Moreover, for the general mathematician, there are many real world factors to consider before selecting a model for formal proof generation, including cost and access to computing power.

We seek to compare several general and special purpose LLMs' capability in formal proof generation. Beyond measuring proof success rates, we also analyze computational cost. Our goal is to provide a clear and practical comparison of different models and strategies, to help identify which approaches are most effective under real-world resource constraints.

Methodology

We construct our benchmark upon proportional subsets of two complementary evaluation datasets: **miniF2F**, which evaluates mathematical reasoning on isolated, competition-level problems (e.g., IMO, AMC, AIME), and **miniCTX**, which tests the model's ability to operate within a heavily-imported theorem structures based on real-world mathematical work.
We evaluate the models under two distinct generation methods. Our methods are as follows:

- **Pass@*k*:** We generate *n* = 32 independent proofs for each theorem. Pass@*k* estimates the probability that at least one out of *k* independent samples successfully verified in the Lean environment.

$$\text{pass@}k := \mathop{\mathbb{E}}_{\text{Problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

where *c* is the number of correct samples out of the *n* = *32* generations and *k* is the number of solutions intended to be checked.

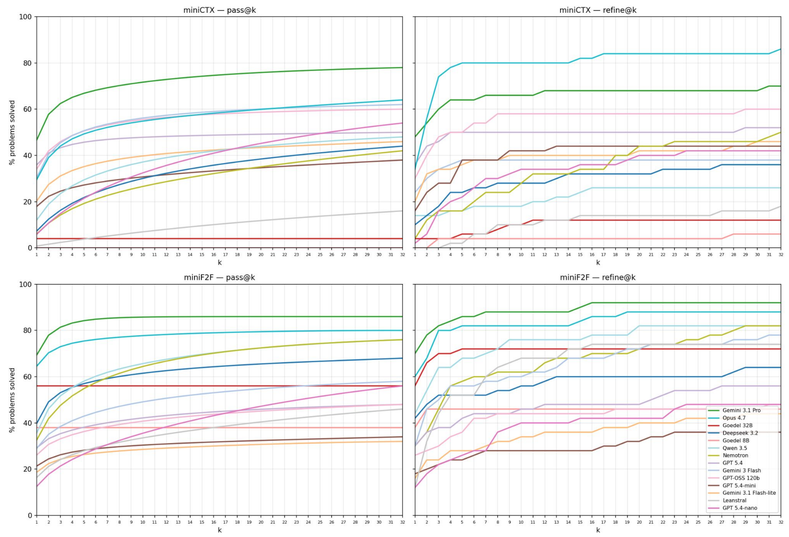
- **Refine@*k*:** If an initial generation fails, the exact trace from the Lean verifier is extracted. The model is re-prompted with its previous faulty proof alongside this specific compiler feedback, iterating for up to *k* turns.

$$\text{Response}_i := \begin{cases} LLM(\text{Prompt}, \text{FTS} := \text{Formal Theorem Statement}) & i = 1 \\ LLM(\text{Prompt}, \text{FTS}, \text{Response}_{i-1}, \text{Feedback}_{i-1}) & 1 < i \leq k \end{cases}$$

General Purpose LLMs: Claude Opus 4.5, GPT 5.4, GPT-5.4-mini, GPT-5.4-nano, Google Gemini 3-flash, Google Gemini 3.1-pro, Google Gemini 3.1-flash-lite, GPT-OSS-120b, Nemotron-3-Super-120b, Qwen3.5-397B, DeepSeek-V3.2

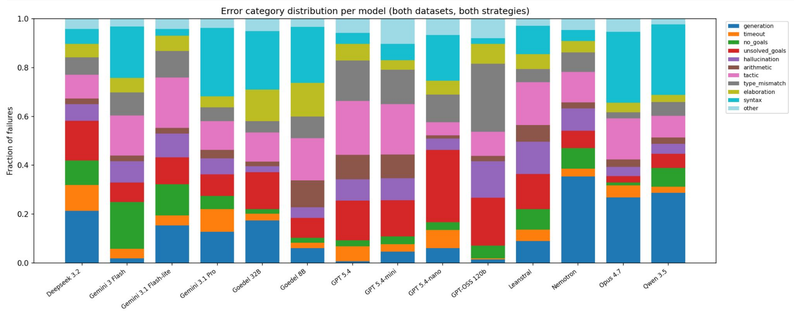
Special Purpose LLMs: Goedel-Prover-32B, Goedel-Prover-8B, Leanstral

Results



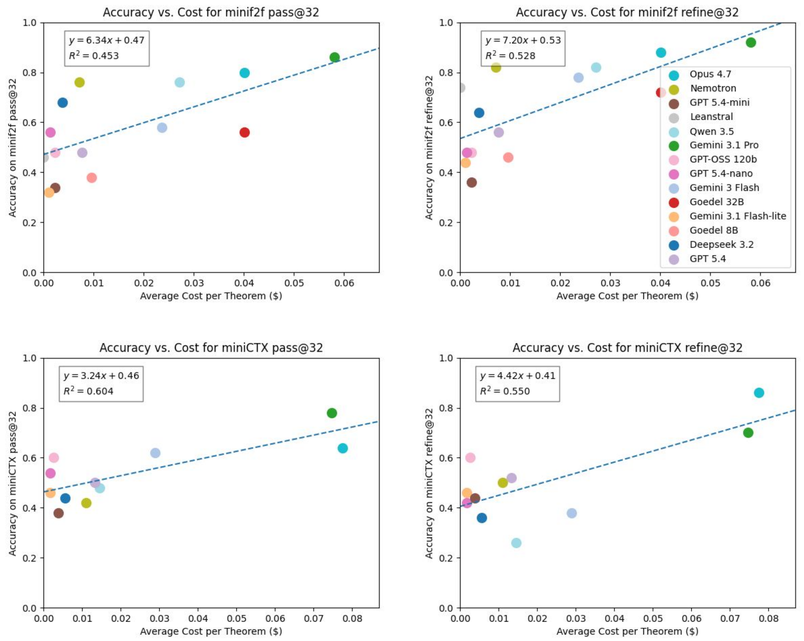
- Opus 4.7 and Gemini 3.1 Pro were the two best-performing models on each method and dataset tested
- Nemotron and Qwen 3.5 performed well on miniF2F, but poorly on miniCTX
- GPT 5.4 and GPT-OSS did relatively poorly on miniF2F, but were among the best in miniCTX
- Specialized models (Goedel, Leanstral) did especially poorly on miniCTX
  - Could be due to an overreliance on short-context fine tuning datasets
- Frontier models show less difference between performance on miniCTX and miniF2F

Results - Errors



- The four dominant error types — tactical errors, syntax errors, unsolved goals, and generation failures — account for 54.7% of all errors.
- Nemotron, Qwen 3.5, and Deepseek 3.2 show high generation failure rates due to runaway responses exceeding token limits.
- Opus fails frequently via 'sorry' and 'admit', suggesting it concedes on hard proofs rather than attempting them.
- GPT 5.4-nano avoids tactical errors but leaves proofs incomplete
- Gemini 3 Flash often continues writing tactics even after goals are already solved.
- miniCTX yields notably more syntax and hallucination errors than miniF2F, as richer context prompts models to fabricate plausible-sounding but nonexistent Mathlib lemmas.

Results - Cost



- There is a strong correlation between cost and accuracy
- We identified the degree of overperformance from expected for each model
  - Nemotron outperformed expected by ~24% on miniF2F but did not notably overperform on miniCTX
  - GPT-OSS overperformed expected by ~15% on miniCTX but underperformed on miniF2F
  - Both these models cost <$0.02 per theorem, making them ideal for mathematicians on a budget

Results - Refine vs. Pass

To analyze the per-model difference between the **pass@*k*** and **refine@*k*** metrics, we define the difference metric

Δ_k := refine@*k* -pass@*k*

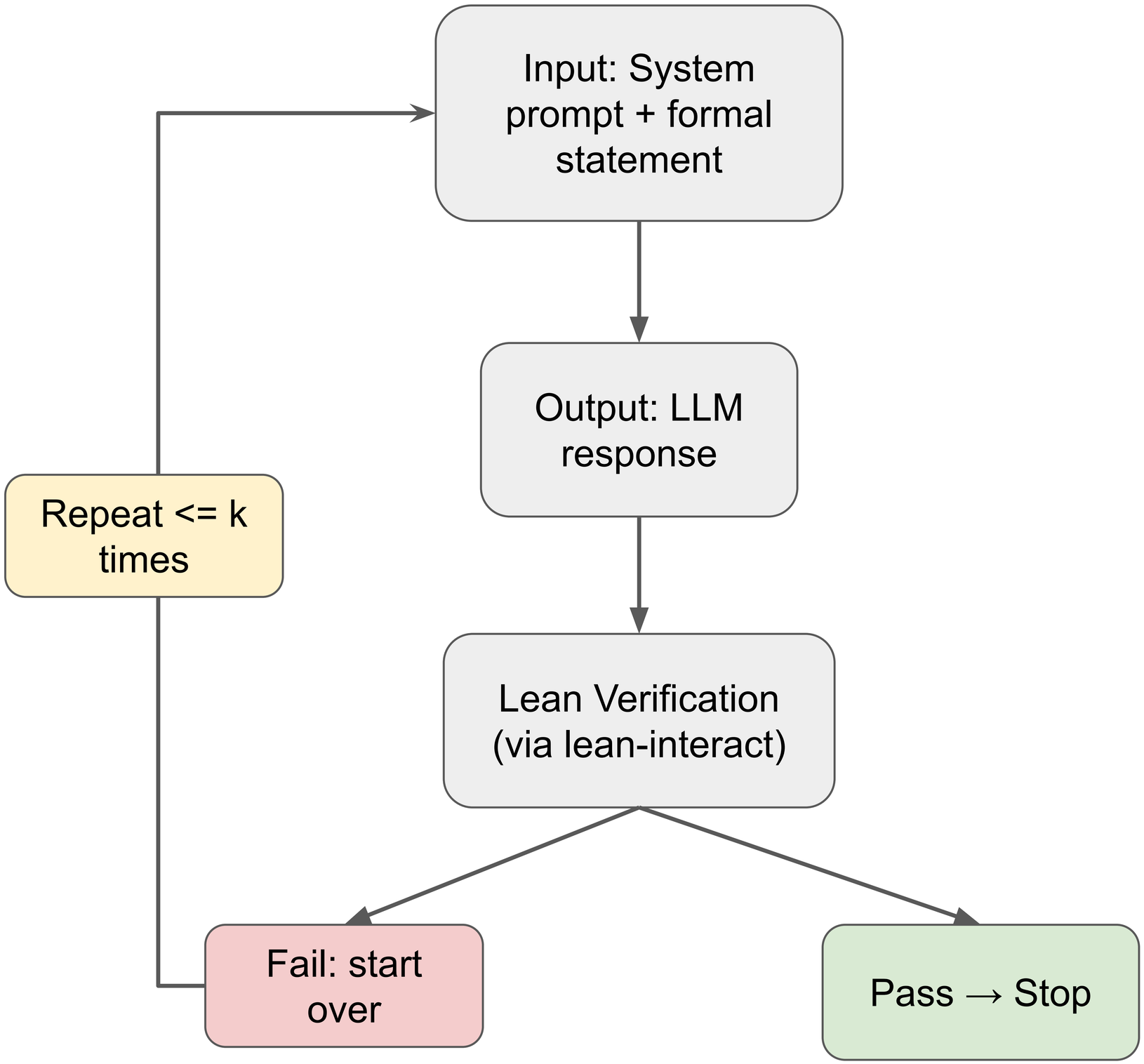
Where a **positive Δ_k** means the model performed better with the iterative refine pipeline compared to the expected value of generating *k* independent responses and a **negative Δ_k** says the opposite.

Δ_{32} on miniF2F and miniCTX

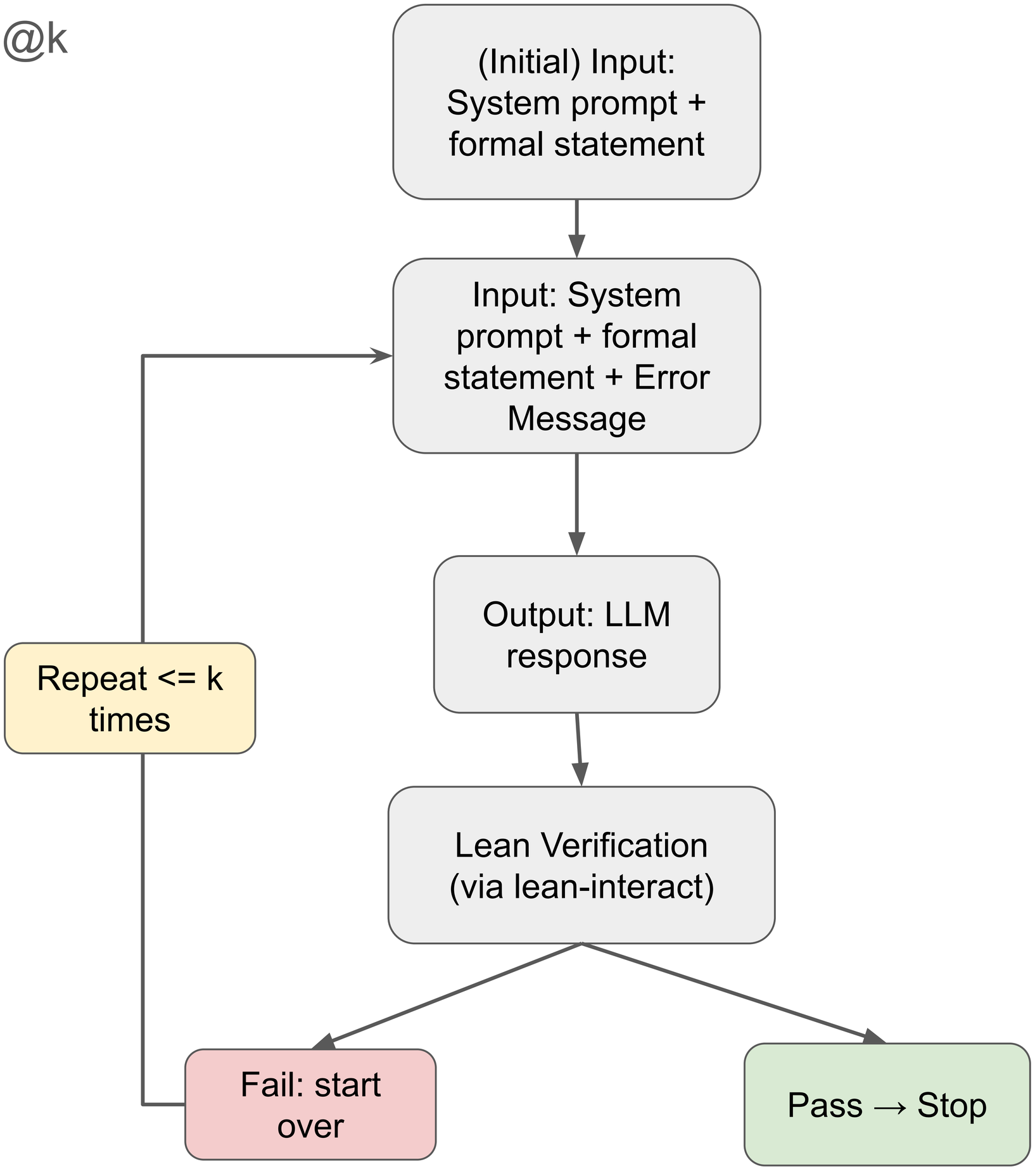
Model	miniF2F	miniCTX
Opus 4.7	8.0	**22.0**
Nemotron	6.0	8.0
Goedel 32B	16.0	8.0
Goedel 8B	8.0	6.0
GPT 5.4-mini	2.0	6.0
GPT 5.4	8.0	2.0
Leanstral	**28.0**	2.0
Gemini Lite	12.0	0.0
GPT-OSS	0.0	0.0
Deepseek	-4.0	-8.0
Gemini Pro	6.0	-8.0
GPT 5.4-nano	-8.0	-12.0
Qwen 3.5	6.0	-22.0
Gemini Flash	20.0	-24.0
---	---	---
Average	**7.7**	**-1.4**

- Most models have a positive Δ_{32} on miniF2F, while less models have a positive Δ_{32} on miniCTX
- Frontier models tend to have higher Δ_{32} overall
  - These models have larger context windows, allowing them to process more information
- miniCTX is more complex, likely leading to the decrease in Δ_{32} for smaller models

Pass@k


Input: System prompt + formal statement
Output: LLM response
Lean Verification (via lean-interact)
Fail: start over
Pass → Stop
Repeat <= k times

Refine@k


(Initial) Input: System prompt + formal statement
Input: System prompt + formal statement + Error Message
Output: LLM response
Lean Verification (via lean-interact)
Fail: start over
Pass → Stop
Repeat <= k times